04_eXtreme_Gradient_Boosting

EP

2026-07-19

eXtreme Gradient Boosting (XGBoost)

```
library(targets)
library(tidymodels)
library(xgboost)
library(sf)
library(dplyr)

set.seed(399)
```

As seen in the Random Forest models, `n_days` is dominant predictor. For species richness, the full model has an excellent ROC AUC (0.956) dropping to 0.706 in the ND model (no `n_days`) which is weaker but moderately useable. However, Simpson diversity models are both weaker. The full model has a moderate ROC AUC of 0.712, dropping to a much weaker 0.662 in the ND model. City level breakdown shows consistency between cities in the full richness model (ROC AUC ranging from 0.981-0.920), whereas the ND richness model has more variabilty (0.795-0.648). This suggests that without the dominant predictor, `n_days`, the model does not generalise to unseen cities as well. Both Simpson models also see greater variation in their ROC AUC scores: full (0.810-0.674) and ND (0.728-0.605). Comparison of the success rate vs prediction rate AUC in final full richness model shows a small decrease (success 0.985 vs prediction 0.956) suggesting the model fits the training data well and can predict unseen data well. However, when removing `n_days` there is a bigger gap (success 0.835 vs prediction 0.706) suggesting more overfitting now the model has to rely on weaker, potentially noisier, signals. Both the Simpson models have larger gaps: full (success 0.792 vs prediction 0.712) and ND (success 0.722 vs prediction 0.662).
However, removing `n_days` highlights `weighted_temp`, `pct_greenspace`, and `weighted_precip` are drivers of richness diversity. `pct_greenspace` and `avg_area_ha` are top drivers of Simpson diversity. This is in agreement with the previous Random Forest results. Despite having lower AUC scores, the ND models are more meaningful when interpreting the ecological drivers of biodiversity.

Load Data

```
setwd("~/CSCT_Pipeline")

grid_model_all <- tar_read(grid_model_all)

df <- grid_model_all %>%
  sf::st_drop_geometry()

n <- nrow(df)
```

```r
df$simpson_adj <- (df$simpson * (n - 1) + 0.5) / n

predictors <- c("n_days", "dtm_elev", "avg_imperv", "rofsw_risk", "rofrs_risk",
                "avg_area_ha", "treeht", "weighted_precip", "weighted_temp",
                "dist_to_river", "pct_greenspace")
```

Model Setup

```r
#Classification

df <- df %>%
  mutate(
    richness_class = factor(if_else(sp_richness >= median(sp_richness, na.rm = TRUE),
                                    "high", "low"),
                            levels = c("low", "high")),
    simpson_class = factor(if_else(simpson_adj >= median(simpson_adj, na.rm = TRUE),
                                   "high", "low"),
                           levels = c("low", "high")))

#XGBoost Specification

xgb_spec_class <- boost_tree(
  trees          = tune(),
  tree_depth     = tune(),
  learn_rate     = tune(),
  loss_reduction = tune(),
  mtry           = tune(),
  min_n          = tune(),
  sample_size    = tune()) %>%
  set_engine("xgboost") %>%
  set_mode("classification")

#Cross-validation folds
folds <- group_vfold_cv(df, group = city)

fold_city_lookup <- folds %>%
  mutate(city = purrr::map_chr(splits, ~ as.character(unique(assessment(.x)$city)))) %>%
  select(id, city)

#Recipes and workflows
richness_recipe_xgb <- recipe(
  as.formula(paste("richness_class ~", paste(predictors, collapse = " + "))),
  data = df) %>%
  step_unknown(all_nominal_predictors()) %>%
  step_naomit(all_numeric_predictors(), all_outcomes()) %>%
  step_novel(all_nominal_predictors()) %>%
  step_dummy(all_nominal_predictors())

richness_wf_xgb <- workflow() %>%
  add_recipe(richness_recipe_xgb) %>%
  add_model(xgb_spec_class)
```

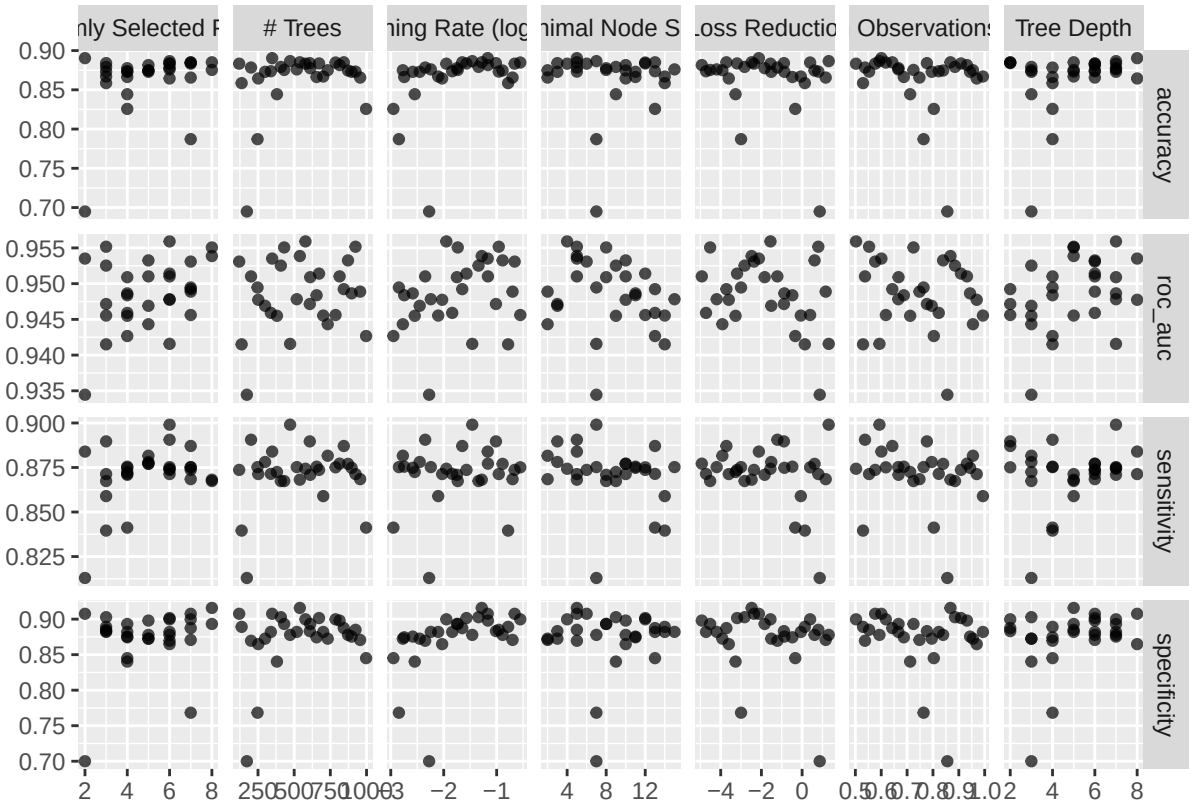
```r
simpson_recipe_xgb <- recipe(
  as.formula(paste("simpson_class ~", paste(predictors, collapse = " + "))),
  data = df) %>%
  step_unknown(all_nominal_predictors()) %>%
  step_naomit(all_numeric_predictors(), all_outcomes()) %>%
  step_novel(all_nominal_predictors()) %>%
  step_dummy(all_nominal_predictors())

simpson_wf_xgb <- workflow() %>%
  add_recipe(simpson_recipe_xgb) %>%
  add_model(xgb_spec_class)
```

Tuning Setup

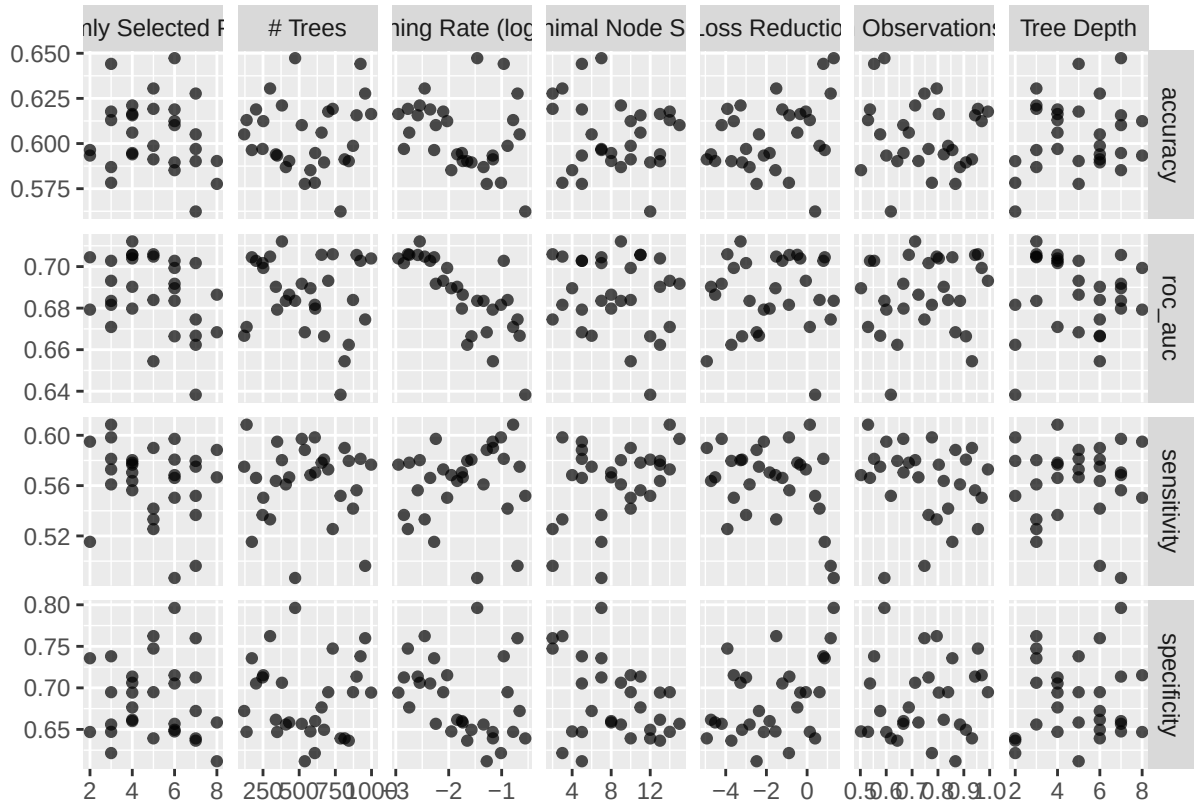
```r
xgb_grid <- grid_space_filling(
  trees(range = c(100, 1000)),
  tree_depth(range = c(2, 8)),
  learn_rate(range = c(-3, -0.5)),   # log10 scale: 0.001 - 0.316
  loss_reduction(range = c(-5, 1.5)),
  finalize(mtry(range = c(2, 8)), df[, predictors]),
  min_n(range = c(2, 15)),
  sample_prop(range = c(0.5, 1.0)),
  size = 30,
  type = "latin_hypercube")
```

Hyperparameter Tuning

```r
richness_res_xgb <- tune_grid(
  richness_wf_xgb,
  resamples = folds,
  grid      = xgb_grid,
  metrics   = metric_set(roc_auc, accuracy, sensitivity, specificity))

simpson_res_xgb <- tune_grid(
  simpson_wf_xgb,
  resamples = folds,
  grid      = xgb_grid,
  metrics   = metric_set(roc_auc, accuracy, sensitivity, specificity))

autoplot(richness_res_xgb)
```




```
autoplot(simpson_res_xgb)
```



Select Best

```
show_best(richness_res_xgb, metric = "roc_auc", n = 5) %>%
  select(mtry, min_n, tree_depth, learn_rate, loss_reduction,
         sample_size, trees, mean, std_err) %>%
  mutate(across(c(mean, std_err), ~ round(.x, 3))) %>%
  rename("Mean ROC AUC" = mean, "Std. error" = std_err) %>%
  knitr::kable(caption = "Top 5 hyperparameter combinations:
               species richness class (XGBoost, ranked by ROC AUC)")
```

Table 1: Top 5 hyperparameter combinations: species richness class (XGBoost, ranked by ROC AUC)

mtry	min_n	tree_depth	learn_rate	loss_reduction	sample_size	trees	Mean ROC AUC	Std. error
6	4	7	0.0112078	0.0279783	0.5026014	578	0.956	0.010
3	5	5	0.1096675	6.3024466	0.5531246	924	0.955	0.013
8	8	5	0.0184371	0.0000302	0.7247171	431	0.955	0.010
8	5	5	0.0523601	0.0033148	0.8677105	539	0.954	0.010
2	5	8	0.0678126	0.0075091	0.6006788	348	0.954	0.009

```r
show_best(simpson_res_xgb, metric = "roc_auc", n = 5) %>%
  select(mtry, min_n, tree_depth, learn_rate, loss_reduction,
         sample_size, trees, mean, std_err) %>%
  mutate(across(c(mean, std_err), ~ round(.x, 3))) %>%
  rename("Mean ROC AUC" = mean, "Std. error" = std_err) %>%
  knitr::kable(caption = "Top 5 hyperparameter combinations:
               Simpson diversity class (XGBoost, ranked by ROC AUC)")
```

Table 2: Top 5 hyperparameter combinations: Simpson diversity class (XGBoost, ranked by ROC AUC)

mtry	min_n	tree_depth	learn_rate	loss_reduction	sample_size	trees	Mean ROC AUC	Std. error
4	9	3	0.0028402	0.0005382	0.7119274	381	0.712	0.027
5	2	3	0.0016925	0.0001210	0.9546309	732	0.706	0.023
4	11	4	0.0017983	0.3320400	0.6873552	655	0.706	0.028
4	11	7	0.0026038	0.1345773	0.9434751	897	0.706	0.020
5	3	3	0.0035202	0.0304874	0.7953771	299	0.705	0.023

```r
best_richness_xgb <- select_best(richness_res_xgb, metric = "roc_auc")
best_simpson_xgb  <- select_best(simpson_res_xgb,  metric = "roc_auc")
```

Prediction Rate AUC

```r
overall_richness_xgb <- collect_metrics(richness_res_xgb) %>%
  filter(mtry == best_richness_xgb$mtry, min_n == best_richness_xgb$min_n,
         tree_depth == best_richness_xgb$tree_depth,
         learn_rate == best_richness_xgb$learn_rate,
         .metric == "roc_auc") %>%
  pull(mean)

overall_simpson_xgb <- collect_metrics(simpson_res_xgb) %>%
  filter(mtry == best_simpson_xgb$mtry, min_n == best_simpson_xgb$min_n,
         tree_depth == best_simpson_xgb$tree_depth,
         learn_rate == best_simpson_xgb$learn_rate,
         .metric == "roc_auc") %>%
  pull(mean)

city_richness_xgb <- collect_metrics(richness_res_xgb, summarize = FALSE) %>%
  filter(mtry == best_richness_xgb$mtry, min_n == best_richness_xgb$min_n,
         tree_depth == best_richness_xgb$tree_depth,
         learn_rate == best_richness_xgb$learn_rate,
         .metric == "roc_auc") %>%
  left_join(fold_city_lookup, by = "id") %>%
  select(city, .estimate) %>%
  rename(richness_auc = .estimate)

city_simpson_xgb <- collect_metrics(simpson_res_xgb, summarize = FALSE) %>%
  filter(mtry == best_simpson_xgb$mtry, min_n == best_simpson_xgb$min_n,
```

```
         tree_depth == best_simpson_xgb$tree_depth,
         learn_rate == best_simpson_xgb$learn_rate,
         .metric == "roc_auc") %>%
  left_join(fold_city_lookup, by = "id") %>%
  select(city, .estimate) %>%
  rename(simpson_auc = .estimate)

city_auc_xgb <- city_richness_xgb %>% full_join(city_simpson_xgb, by = "city")

overall_auc_xgb <- tibble(city = "Overall",
                          richness_auc = overall_richness_xgb,
                          simpson_auc  = overall_simpson_xgb)

classification_auc_table_xgb <- bind_rows(overall_auc_xgb, city_auc_xgb) %>%
  mutate(across(where(is.numeric), ~ round(.x, 3))) %>%
  rename("City (Held-Out)" = city,
         "Richness ROC AUC" = richness_auc,
         "Simpson ROC AUC"  = simpson_auc)

classification_auc_table_xgb %>%
  knitr::kable(caption = "Overall and city-level cross-validated ROC AUC for
               species richness and Simpson diversity (XGBoost)")
```

Table 3: Overall and city-level cross-validated ROC AUC for species richness and Simpson diversity (XGBoost)

City (Held-Out)	Richness ROC AUC	Simpson ROC AUC
Overall	0.956	0.712
Nottingham	0.920	0.674
Exeter	0.950	0.701
Newcastle	0.963	0.658
Bristol	0.981	0.717
Gateshead	0.967	0.810

Final Model

```
final_richness_wf_xgb <- finalize_workflow(richness_wf_xgb, best_richness_xgb)
richness_fit_xgb <- fit(final_richness_wf_xgb, data = df)

final_simpson_wf_xgb <- finalize_workflow(simpson_wf_xgb, best_simpson_xgb)
simpson_fit_xgb <- fit(final_simpson_wf_xgb, data = df)
```

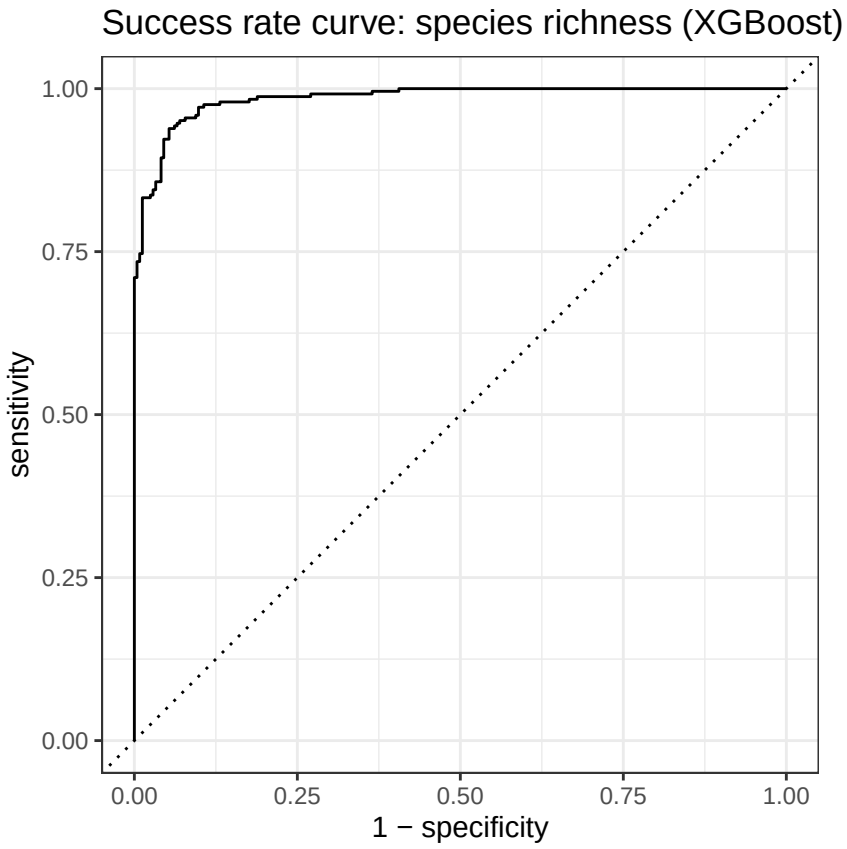
Success Rate AUC

```
richness_success_xgb <- augment(richness_fit_xgb, new_data = df)
simpson_success_xgb  <- augment(simpson_fit_xgb,  new_data = df)

success_rate_richness_xgb <- roc_auc(richness_success_xgb, truth = richness_class,
```

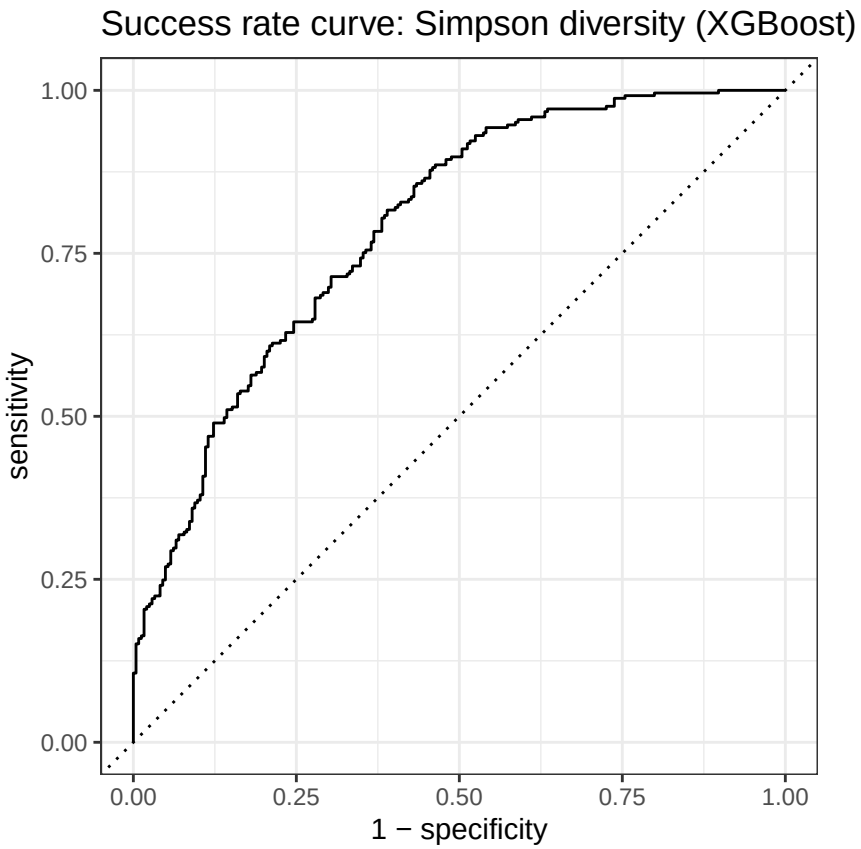
```
                                        .pred_high, event_level = "second")
success_rate_simpson_xgb <- roc_auc(simpson_success_xgb, truth = simpson_class,
                                    .pred_high, event_level = "second")
```

ROC Curves

```
roc_curve(richness_success_xgb, truth = richness_class, .pred_high, event_level = "second")%>%
  autoplot() + labs(title = "Success rate curve: species richness (XGBoost)")
```




```
roc_curve(simpson_success_xgb, truth = simpson_class, .pred_high, event_level = "second")%>%
  autoplot() + labs(title = "Success rate curve: Simpson diversity (XGBoost)")
```



Summary Table

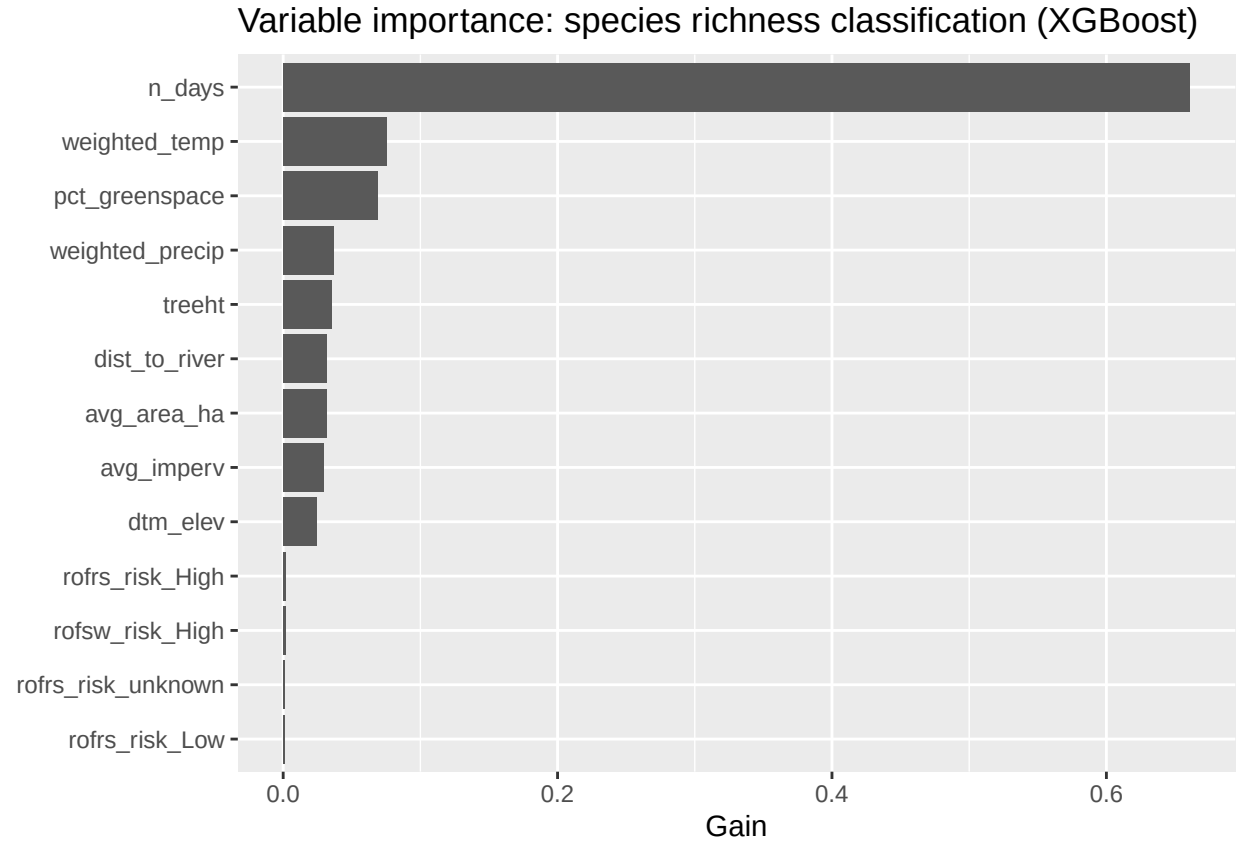
```
tibble::tibble(
  Outcome = c("Species richness", "Simpson diversity"),
  `Success rate AUC` = c(success_rate_richness_xgb$.estimate,
                         success_rate_simpson_xgb$.estimate),
  `Prediction rate AUC` = c(overall_richness_xgb, overall_simpson_xgb)
) %>%
  mutate(across(where(is.numeric), ~ round(.x, 3))) %>%
  knitr::kable(caption = "XGBoost classification performance:
               success rate vs. prediction rate AUC")
```

Table 4: XGBoost classification performance: success rate vs. prediction rate AUC

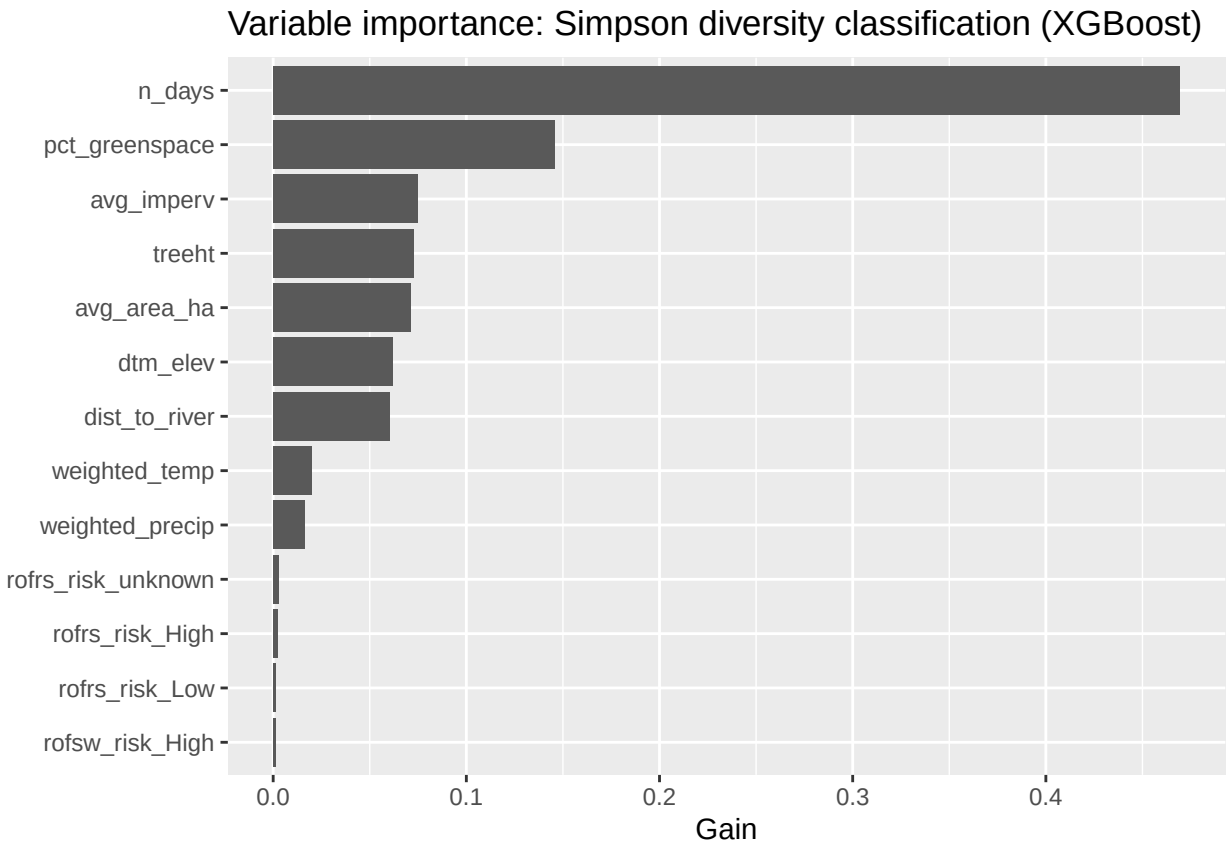
Outcome	Success rate AUC	Prediction rate AUC
Species richness	0.985	0.956
Simpson diversity	0.790	0.712

Variable Importance

```
richness_fit_xgb %>%
  extract_fit_engine() %>%
  xgboost::xgb.importance(model = .) %>%
  as.data.frame() %>%
  ggplot(aes(x = reorder(Feature, Gain), y = Gain)) +
  geom_col() +
  coord_flip() +
  labs(title = "Variable importance: species richness classification (XGBoost)",
       x = NULL, y = "Gain")
```




```
simpson_fit_xgb %>%
  extract_fit_engine() %>%
  xgboost::xgb.importance(model = .) %>%
  as.data.frame() %>%
  ggplot(aes(x = reorder(Feature, Gain), y = Gain)) +
  geom_col() +
  coord_flip() +
  labs(title = "Variable importance: Simpson diversity classification (XGBoost)",
       x = NULL, y = "Gain")
```

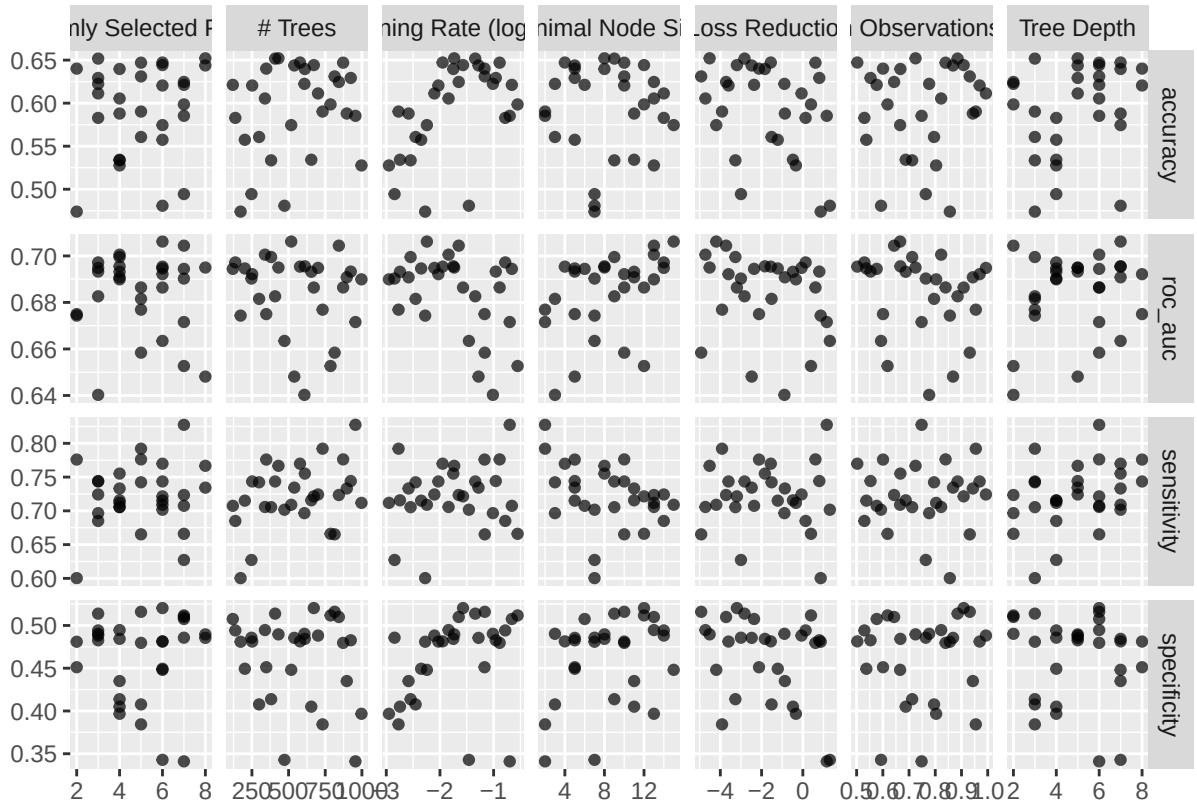


XGBoost Without `n_days`

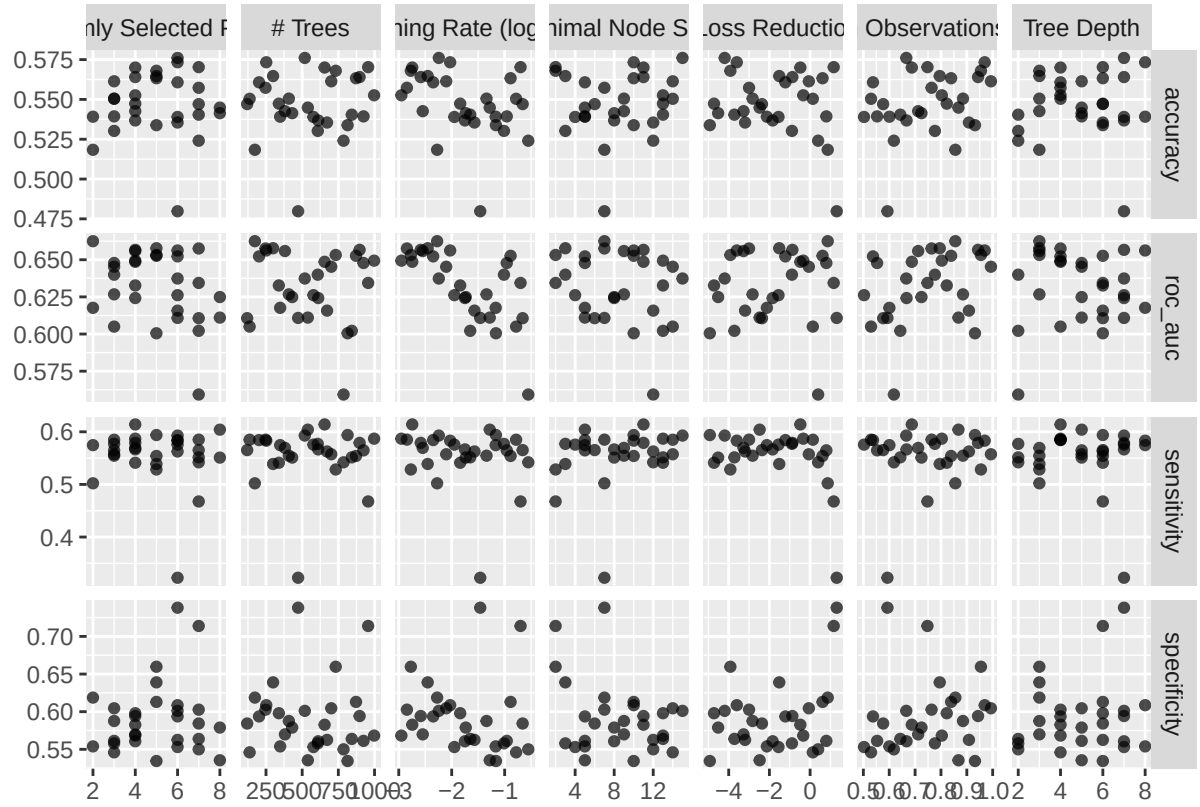
Set up and tuning

```
predictors_ND <- c("dtm_elev", "avg_imperv", "rofsw_risk", "rofrs_risk",
                   "avg_area_ha", "treeht", "weighted_precip", "weighted_temp",
                   "dist_to_river", "pct_greenspace")
#Recipes and workflows
richness_recipe_xgb_ND <- recipe(
  as.formula(paste("richness_class ~", paste(predictors_ND, collapse = " + "))),
  data = df) %>%
  step_unknown(all_nominal_predictors()) %>%
  step_naomit(all_numeric_predictors(), all_outcomes()) %>%
  step_novel(all_nominal_predictors()) %>%
  step_dummy(all_nominal_predictors())

richness_wf_xgb_ND <- workflow() %>%
  add_recipe(richness_recipe_xgb_ND) %>%
  add_model(xgb_spec_class)

simpson_recipe_xgb_ND <- recipe(
  as.formula(paste("simpson_class ~", paste(predictors_ND, collapse = " + "))),
  data = df) %>%
```

```
  step_unknown(all_nominal_predictors()) %>%
  step_naomit(all_numeric_predictors(), all_outcomes()) %>%
  step_novel(all_nominal_predictors()) %>%
  step_dummy(all_nominal_predictors())

simpson_wf_xgb_ND <- workflow() %>%
  add_recipe(simpson_recipe_xgb_ND) %>%
  add_model(xgb_spec_class)

#Hyperparameter Tuning
richness_res_xgb_ND <- tune_grid(
  richness_wf_xgb_ND,
  resamples = folds,
  grid      = xgb_grid,
  metrics   = metric_set(roc_auc, accuracy, sensitivity, specificity))

simpson_res_xgb_ND <- tune_grid(
  simpson_wf_xgb_ND,
  resamples = folds,
  grid      = xgb_grid,
  metrics   = metric_set(roc_auc, accuracy, sensitivity, specificity))

autoplot(richness_res_xgb_ND)
```



```
autoplot(simpson_res_xgb_ND)
```




```
#Select Best
show_best(richness_res_xgb_ND, metric = "roc_auc", n = 5) %>%
  select(mtry, min_n, tree_depth, learn_rate, loss_reduction,
         sample_size, trees, mean, std_err) %>%
  mutate(across(c(mean, std_err), ~ round(.x, 3))) %>%
  rename("Mean ROC AUC" = mean, "Std. error" = std_err) %>%
  knitr::kable(caption = "Top 5 hyperparameter combinations:
               species richness ND class (XGBoost, ranked by ROC AUC)")
```

Table 5: Top 5 hyperparameter combinations: species richness ND class (XGBoost, ranked by ROC AUC)

mtry	min_n	tree_depth	learn_rate	loss_reduction	sample_size	trees	Mean ROC AUC	Std. error
6	15	7	0.0057096	0.0000636	0.6659450	518	0.706	0.026
7	13	2	0.0223982	0.0001897	0.6429801	843	0.704	0.033
4	13	6	0.0144742	0.0000189	0.8224535	339	0.701	0.033
4	9	3	0.0028402	0.0005382	0.7119274	381	0.700	0.030
3	14	4	0.1647019	1.3640763	0.5297650	137	0.697	0.026

```r
show_best(simpson_res_xgb_ND, metric = "roc_auc", n = 5) %>%
  select(mtry, min_n, tree_depth, learn_rate, loss_reduction,
         sample_size, trees, mean, std_err) %>%
  mutate(across(c(mean, std_err), ~ round(.x, 3))) %>%
  rename("Mean ROC AUC" = mean, "Std. error" = std_err) %>%
  knitr::kable(caption = "Top 5 hyperparameter combinations:
               Simpson diversity ND class (XGBoost, ranked by ROC AUC)")
```

Table 6: Top 5 hyperparameter combinations: Simpson diversity ND class (XGBoost, ranked by ROC AUC)

mtry	min_n	tree_depth	learn_rate	loss_reduction	sample_size	trees	Mean ROC AUC	Std. error
2	7	3	0.0053055	7.3573837	0.8556851	173	0.662	0.026
5	3	3	0.0035202	0.0304874	0.7953771	299	0.658	0.014
7	7	4	0.0014263	0.0009986	0.7637526	247	0.658	0.018
4	11	7	0.0026038	0.1345773	0.9434751	897	0.657	0.014
6	10	8	0.0093492	0.0002497	0.9692056	252	0.656	0.014

```r
best_richness_xgb_ND <- select_best(richness_res_xgb_ND, metric = "roc_auc")
best_simpson_xgb_ND  <- select_best(simpson_res_xgb_ND,  metric = "roc_auc")
```

Prediction Rate AUC

```r
overall_richness_xgb_ND <- collect_metrics(richness_res_xgb_ND) %>%
  filter(mtry == best_richness_xgb_ND$mtry, min_n == best_richness_xgb_ND$min_n,
         tree_depth == best_richness_xgb_ND$tree_depth,
         learn_rate == best_richness_xgb_ND$learn_rate,
         .metric == "roc_auc") %>%
  pull(mean)

overall_simpson_xgb_ND <- collect_metrics(simpson_res_xgb_ND) %>%
  filter(mtry == best_simpson_xgb_ND$mtry, min_n == best_simpson_xgb_ND$min_n,
         tree_depth == best_simpson_xgb_ND$tree_depth,
         learn_rate == best_simpson_xgb_ND$learn_rate,
         .metric == "roc_auc") %>%
  pull(mean)

city_richness_xgb_ND <- collect_metrics(richness_res_xgb_ND, summarize = FALSE) %>%
  filter(mtry == best_richness_xgb_ND$mtry, min_n == best_richness_xgb_ND$min_n,
         tree_depth == best_richness_xgb_ND$tree_depth,
         learn_rate == best_richness_xgb_ND$learn_rate,
         .metric == "roc_auc") %>%
  left_join(fold_city_lookup, by = "id") %>%
  select(city, .estimate) %>%
  rename(richness_auc = .estimate)

city_simpson_xgb_ND <- collect_metrics(simpson_res_xgb_ND, summarize = FALSE) %>%
  filter(mtry == best_simpson_xgb_ND$mtry, min_n == best_simpson_xgb_ND$min_n,
```

```
        tree_depth == best_simpson_xgb_ND$tree_depth,
        learn_rate == best_simpson_xgb_ND$learn_rate,
        .metric == "roc_auc") %>%
  left_join(fold_city_lookup, by = "id") %>%
  select(city, .estimate) %>%
  rename(simpson_auc = .estimate)

city_auc_xgb_ND <- city_richness_xgb_ND %>% full_join(city_simpson_xgb_ND, by = "city")

overall_auc_xgb_ND <- tibble(city = "Overall",
                             richness_auc = overall_richness_xgb_ND,
                             simpson_auc  = overall_simpson_xgb_ND)

classification_auc_table_xgb_ND <- bind_rows(overall_auc_xgb_ND, city_auc_xgb_ND) %>%
  mutate(across(where(is.numeric), ~ round(.x, 3))) %>%
  rename("City (Held-Out)" = city,
         "Richness ROC AUC" = richness_auc,
         "Simpson ROC AUC"  = simpson_auc)

classification_auc_table_xgb_ND %>%
  knitr::kable(caption = "Overall and city-level cross-validated ROC AUC for
               ND species richness and Simpson diversity (XGBoost)")
```

Table 7: Overall and city-level cross-validated ROC AUC for ND species richness and Simpson diversity (XGBoost)

City (Held-Out)	Richness ROC AUC	Simpson ROC AUC
Overall	0.706	0.662
Nottingham	0.693	0.609
Exeter	0.648	0.656
Newcastle	0.663	0.605
Bristol	0.795	0.715
Gateshead	0.732	0.728

Final Model

```
final_richness_wf_xgb_ND <- finalize_workflow(richness_wf_xgb_ND, best_richness_xgb_ND)
richness_fit_xgb_ND <- fit(final_richness_wf_xgb_ND, data = df)

final_simpson_wf_xgb_ND <- finalize_workflow(simpson_wf_xgb_ND, best_simpson_xgb_ND)
simpson_fit_xgb_ND <- fit(final_simpson_wf_xgb_ND, data = df)
```

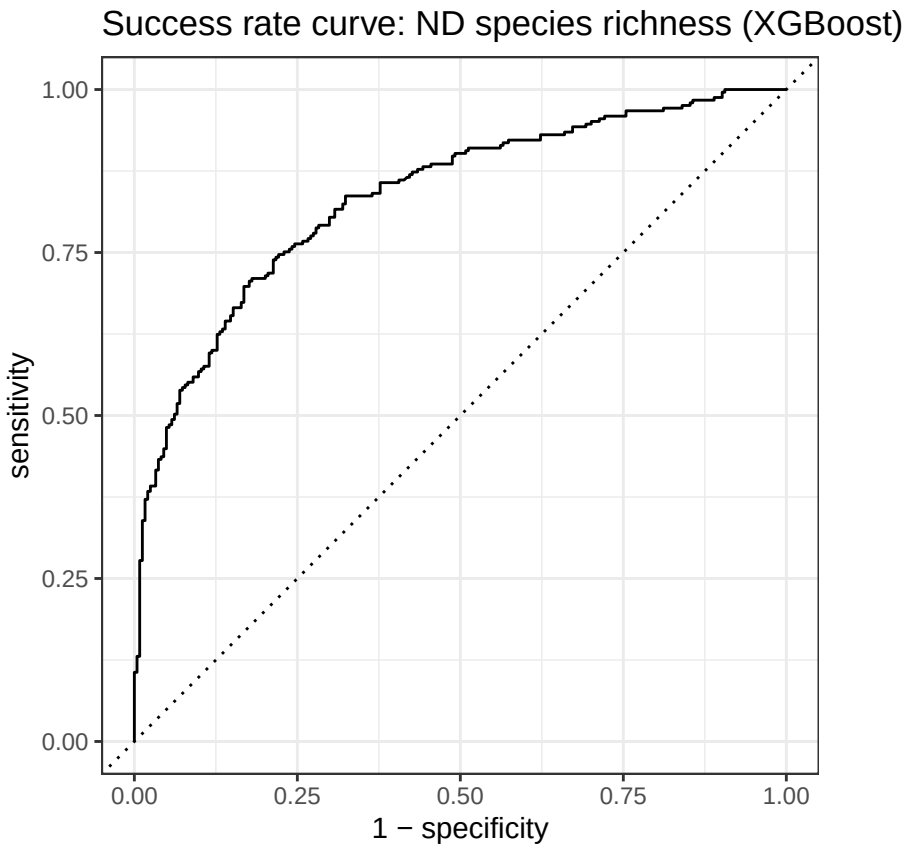
Success Rate AUC

```
richness_success_xgb_ND <- augment(richness_fit_xgb_ND, new_data = df)
simpson_success_xgb_ND  <- augment(simpson_fit_xgb_ND,  new_data = df)

success_rate_richness_xgb_ND <- roc_auc(richness_success_xgb_ND, truth = richness_class,
```

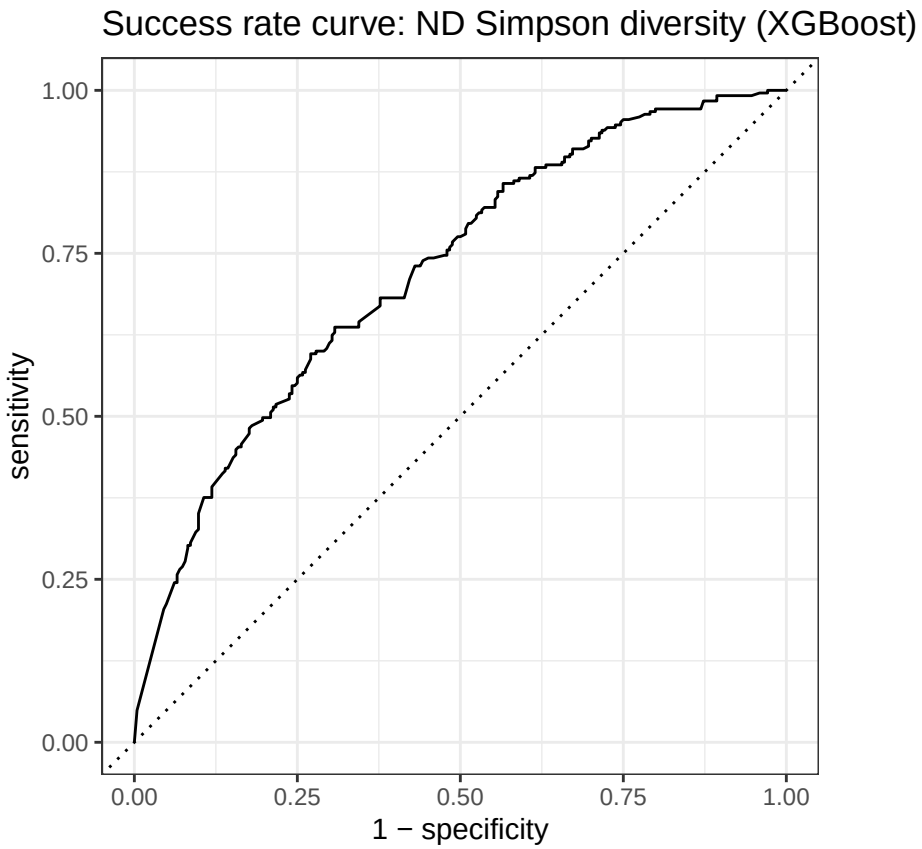
```
                                        .pred_high, event_level = "second")
success_rate_simpson_xgb_ND <- roc_auc(simpson_success_xgb_ND, truth = simpson_class,
                                       .pred_high, event_level = "second")
```

ROC Curves

```
roc_curve(richness_success_xgb_ND, truth = richness_class, .pred_high,
          event_level = "second") %>%
  autoplot() + labs(title = "Success rate curve: ND species richness (XGBoost)")
```




```
roc_curve(simpson_success_xgb_ND, truth = simpson_class, .pred_high,
          event_level = "second") %>%
  autoplot() + labs(title = "Success rate curve: ND Simpson diversity (XGBoost)")
```



Summary Table

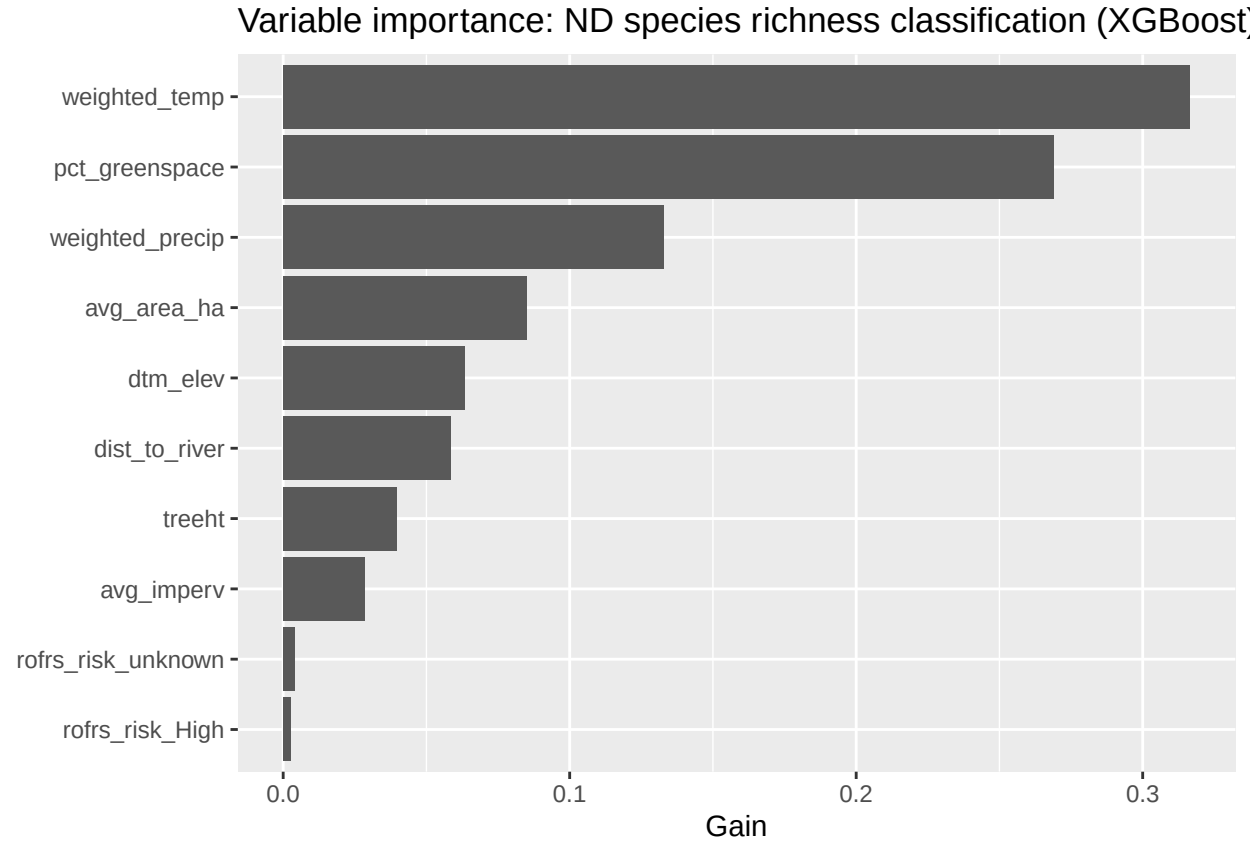
```
tibble::tibble(
  Outcome = c("ND Species richness", "ND Simpson diversity"),
  `Success rate AUC` = c(success_rate_richness_xgb_ND$.estimate,
                         success_rate_simpson_xgb_ND$.estimate),
  `Prediction rate AUC` = c(overall_richness_xgb_ND, overall_simpson_xgb_ND)
) %>%
  mutate(across(where(is.numeric), ~ round(.x, 3))) %>%
  knitr::kable(caption = "XGBoost classification performance:
               ND success rate vs. ND prediction rate AUC")
```

Table 8: XGBoost classification performance: ND success rate vs. ND prediction rate AUC

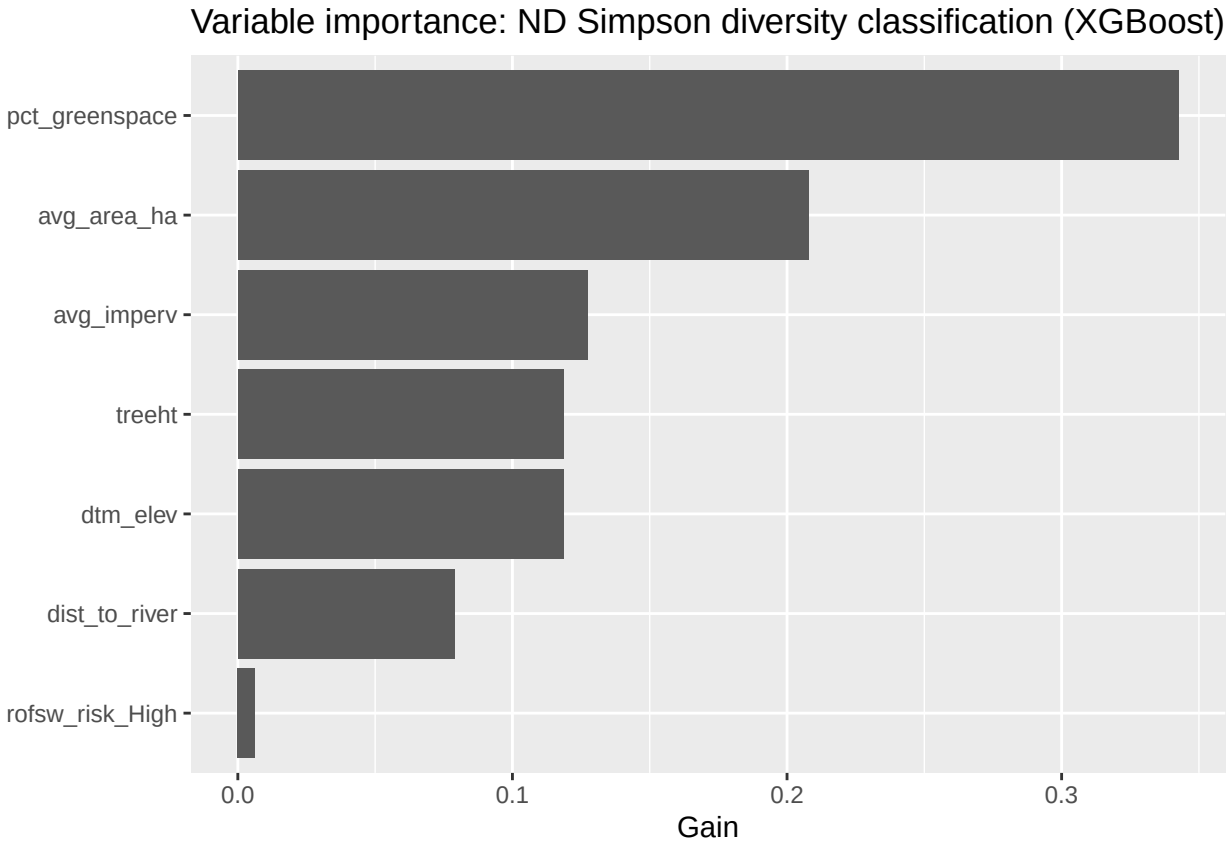
Outcome	Success rate AUC	Prediction rate AUC
ND Species richness	0.835	0.706
ND Simpson diversity	0.722	0.662

Variable Importance

```
richness_fit_xgb_ND %>%
  extract_fit_engine() %>%
  xgboost::xgb.importance(model = .) %>%
  as.data.frame() %>%
  ggplot(aes(x = reorder(Feature, Gain), y = Gain)) +
  geom_col() +
  coord_flip() +
  labs(title = "Variable importance: ND species richness classification (XGBoost)",
       x = NULL, y = "Gain")
```




```
simpson_fit_xgb_ND %>%
  extract_fit_engine() %>%
  xgboost::xgb.importance(model = .) %>%
  as.data.frame() %>%
  ggplot(aes(x = reorder(Feature, Gain), y = Gain)) +
  geom_col() +
  coord_flip() +
  labs(title = "Variable importance: ND Simpson diversity classification (XGBoost)",
       x = NULL, y = "Gain")
```


Variable importance: ND Simpson diversity classification (XGBoost)
pct_greenspace
avg_area_ha
avg_imperv
treeht
dtm_elev
dist_to_river
rofsw_risk_High
0.0
0.1
0.2
0.3
Gain